On Relativizations of BPP

Pico Gilman

May 2026

Abstract

We describe a simplified proof of the existence of a relativized world where $\mathbf{EXP}^{\mathbf{NP}} \subseteq \mathbf{BPP}$. Combined with the relativized proof of $\mathbf{BPP} \subseteq \Sigma_2\mathbf{P}$, we provide a short proof that there is a relativization where $\mathbf{BPP} = \Sigma_2\mathbf{P} = \mathbf{EXP}^{\mathbf{NP}}$.

Contents

1 Introduction

Deterministic Turing machines provide a powerful tool for computing inclusion in some potentially otherwise complicated family. In addition, many problems seem to have structural advantages when using randomness. For example, the Miller-Rabin primality test, which on an input x of length n, computes whether x is prime in $O(n^3)$ time and $O(n)$ coin flips with error bounded by $1/3$, was only discovered in 1980 [Rab80]. In contrast, it took until 2004 for an unconditional deterministic polynomial time algorithm for computing whether or not x is prime to be discovered [AKS04], and even then it had a time complexity of $O(n^{21/2+\epsilon})$, which was later improved to $O(n^{6+\epsilon})$ [LP19]. Even assuming the Generalized Riemann Hypothesis, this only improves to $O(n^{4+\epsilon})$ [Bac90].

In addition, there are many natural problems which have nearly trivial probabilistic polynomial time algorithms such as polynomial identity checking that have no known deterministic polynomial time algorithm:

Problem (Polynomial Identity Testing)**.** Given some circuit that computes some polynomial $f \in \mathbb{Z}[x_1, \ldots, x_n]$, is f the 0 polynomial?

Specifically, pick a prime $p \gg \deg f$, and compute $f(r_1, \ldots, r_n)$ where $r_i \leftarrow \mathbb{Z}/p\mathbb{Z}$ independently and randomly chosen. The probability that $f(r_1, \ldots, r_n) = 0$ when it is not the 0 polynomial is then bounded by $\frac{\deg f}{p}$, which can be made arbitrarily small [Sch80].

With this motivation, we first will define what we mean by probabilistic polynomial time, and introduce a few related notions.

Definition 1.1. A language $L \subseteq \{0,1\}^*$ is in **BPP** if there exists a probabilistic polynomial-time Turing machine M such that for all inputs x:

$$\begin{aligned} x \in L &\implies \Pr[M(x) = 1] \geq \tfrac{2}{3}, \\ x \notin L &\implies \Pr[M(x) = 0] \geq \tfrac{2}{3}. \end{aligned}$$

Equivalently, since M runs in polynomial time, we can find an explicit bound $C|x|^e$ for any input x of the number of coin flips used. If $M(x, r)$ denotes M ran on input x with r as the randomness, then we require

$$\begin{aligned} x \in L &\implies \frac{1}{2^{C|x|^e}} \#\{r : M(x,r) \text{ accepts}\} \geq \tfrac{2}{3}, \\ x \notin L &\implies \frac{1}{2^{C|x|^e}} \#\{r : M(x,r) \text{ rejects}\} \geq \tfrac{2}{3}. \end{aligned}$$

It is trivial to show $\mathbf{P} \subseteq \mathbf{BPP} \subseteq \mathbf{EXP}$. Neither inclusion is known to be strict, and in this paper we study the power of **BPP** by understanding its properties in some relativization. For example, Baker, Gill, and Solovay proved the following theorem.

Theorem 1.2 ([BGS75], Theorem 1, Theorem 3)**.** There are oracles $\mathbf{B}$ and $\mathbf{B}'$ such that

$$\mathbf{P}^{\mathbf{B}} = \mathbf{NP}^{\mathbf{B}} \quad \text{and} \quad \mathbf{P}^{\mathbf{B}'} \neq \mathbf{NP}^{\mathbf{B}'}.$$

Consequently, any proof resolving $\mathbf{P} = \mathbf{NP}$ must use non-relativizing techniques. Similarly, we will show that any proofs of $\mathbf{EXP}^{\mathbf{NP}} \neq \mathbf{BPP}$ have to be non-relativizing.

We also note that Bennett and Gill [BG81] later proved that by picking a random oracle $\mathbf{A}$, with probability 1, we have

- $\mathbf{BPP}^{\mathbf{A}} = \mathbf{P}^{\mathbf{A}}$
- $\mathbf{NP}^{\mathbf{A}} \supsetneq \mathbf{P}^{\mathbf{A}}$,

and therefore $\mathbf{BPP}^{\mathbf{A}} \subsetneq \mathbf{NP}^{\mathbf{A}} \subseteq \Sigma_2\mathbf{P}^{\mathbf{A}}$. Thus, any proof showing agreement or disagreement between **BPP**, $\Sigma_2\mathbf{P}$, and $\mathbf{EXP}^{\mathbf{NP}}$ would have to be non-relativizing.

1.1 Organization and notation

Throughout the paper $\mathbf{A}$ will denote an oracle. Our alphabet is $\{0,1\}$. The languages $\mathbf{NP}, \mathbf{P}, \mathbf{EXP}$ and $\Sigma_2\mathbf{P}$ all take on their standard definitions.

Section 2 proves[1] that $\mathbf{BPP} \subseteq \Sigma_2\mathbf{P}$ relativizes. To prove that the equality can hold, we need Proposition 3.3 which shows **EXP** using a **NP** oracle is weaker than getting polynomial advice implies said **NP** oracle does not even help. Assuming an oracle with certain properties exist, we then finish the proof of Theorem 3.1 in Section 3.2. Then, in Section 4, we prove Proposition 3.2, which constructs an oracle with our desired properties.

2 Warm up: Relativizable Proof of $\mathbf{BPP} \subseteq \Sigma_2\mathbf{P}$

The goal of this section is to prove that the proof of $\mathbf{BPP} \subseteq \Sigma_2\mathbf{P}$ relativizes.

Lemma 2.1. Let $p(n) > 3n$ be some polynomial and $n > 9$. Let $S \subseteq \{0,1\}^{p(n)} = \mathbb{F}_2^{p(n)} = V$ have density $> 1 - 2^{-n}$. Then there are $p(n)$ affine translates of S, say $z_i + S$ for $i = 1, \ldots, p(n)$, that together cover V.

Proof. It suffices to show that a random selection of z_i works with positive probability, which will tell us that some choice exists.

Let $v \in V$. The chance that any single translate $z_i + S$ does not contain v is at most 2^{-n}. Thus, the chance that $v \notin \cup(z_i + S)$ is at most $2^{-np(n)}$. By a union bound and linearity of expectation, the probability that $\cup(z_i + S) \neq V$ is bounded by $\#V 2^{-np(n)} < \frac{1}{2}$, and thus we are done. □

With this, we can prove this section's main theorem.

Theorem 2.2. For any oracle $\mathbf{A}$, we have $\mathbf{BPP}^{\mathbf{A}} \subseteq (\Sigma_2 P)^{\mathbf{A}} = \mathbf{NP}^{\mathbf{NP}^{\mathbf{A}}}$.

Proof. Let $L \in \mathbf{BPP}^{\mathbf{A}}$. There is some Turing machine T that decides $x \in L$ with $C \cdot |x|^e$ bits of randomness with probability at least $2/3$. Let T' be the algorithm that runs this algorithm $10n+1$ times and accepting exactly when the majority of the runs accept, we error with probability bounded by

$$\sum_{i=5n+1}^{10n+1} \left(\frac{1}{3}\right)^i \left(\frac{2}{3}\right)^{10n+1-i} \binom{10n+1}{i} \leq 6n3^{-10n-1}2^{10n+1}2^{5n} \leq 4n\left(\frac{8}{9}\right)^{5n},$$

which for $n > 10$ is bounded by $2^{-n/10}$.

Thus, consider the set of $B = (100|x|+1)C|x|^e$-length strings. Of the 2^B of these randomness possibilities, T' accepts density $< 2^{-|x|}$ of these when $x \notin L$ and accepts with density $> 1 - 2^{-|x|}$ when $x \in L$.

We are now ready to describe our algorithm so that $L \in (\Sigma_2\mathbf{P})^{\mathbf{A}}$. First, existentially guess B-many strings of length B, call them $z_1, \ldots, z_B$. Then, for each $y \in \{0,1\}^B$, check if at least one of $T'(x, y \oplus z_i)$ comes to true. If so, accept, if not reject.

Suppose $x \in L$. Then by Lemma 2.1 there is some choice of z_i that will suffice. Conversely, if $x \notin L$ then at most density $B2^{-|x|} < 1$ of the strings $y \in \{0,1\}^B$ will cause T' to accept, and therefore we reject (n.b. the $|x|$ such that $B2^{-|x|} \geq 1$ is a finite set and therefore every possible randomness input to L can be tested, upon which we can correctly output the desired answer). Thus, T' witnesses $L \in (\Sigma_2 P)^{\mathbf{A}}$. □

[1] Following the idea of Ryan Williams' 18.405 Lecture on March 19, 2026.

3 A relativized world where $\mathbf{EXP}^{\mathbf{NP}} \subseteq \mathbf{BPP}$

The goal of this section is to prove the following theorem.

Theorem 3.1. There is some oracle $\mathbf{A}$ such that $\mathbf{EXP}^{\mathbf{NP^A}} \subseteq \mathbf{BPP}^{\mathbf{A}}$.

Before proving this theorem, we need to construct a sufficient strong oracle as to allow a massive amount of collapse. The following proposition will suffice.

Proposition 3.2. There is some oracle $\mathbf{A}$ such that $\mathbf{EXP}^{\mathbf{NP^A}} \subseteq \mathbf{NP}^{\mathbf{A}}/\text{poly}$ and $\oplus\mathbf{P}^{\mathbf{A}} = \mathbf{P}^{\mathbf{A}}$.

This proof is technical, and due to length reasons, is proven in Section 4.

3.1 Connecting Lemma

We need a proposition that relates various classes to others. It shows that a **NP** oracle is not useful with exponential time if polynomial advice is sufficient for **NP** calculations, and does so by effectively guessing the advice string.

Proposition 3.3 (Buhrman and Homer, [BH92] §4)**.** For any oracle $\mathbf{A}$, if $\mathbf{EXP}^{\mathbf{NP^A}} \subseteq \mathbf{EXP}^{\mathbf{A}}/\text{poly}$ then $\mathbf{EXP}^{\mathbf{NP^A}} = \mathbf{EXP}^{\mathbf{A}}$.

We loosely follow the idea of [BH92]. The idea of this proof is to find a $\mathbf{EXP}^{\mathbf{NP^A}}$-complete problem and show it is in $\mathbf{EXP}^{\mathbf{A}}$ by finding the correct advice string. Motivated by "does the Turing machine T halt within t steps", where you write t in binary is an **EXP**-complete problem, one can show the following statement.

Lemma 3.4. Let $L_1, L_2, \ldots,$ be some enumeration of the languages in $\mathbf{NEXP}^{\mathbf{A}}$ solved by oracle Turing machines $M_1, M_2 \ldots$. Put

$$L = \left\{ (n, x, t, i, b) \text{ such that } \begin{array}{c} \text{(a) } M_n^{\mathbf{A}}(x) \text{ accepts within } t \text{ steps, and (b) of all} \\ \text{accepting paths, the one that is lexicographically} \\ \text{smallest has } b \text{ as the } i\text{th bit in its computation path} \end{array} \right\},$$

where $n, t, i \in \mathbb{N}$, written in binary, $x \in \{0,1\}^*$, and $b \in \{0,1\}$. Then L is $\mathbf{EXP}^{\mathbf{NP^A}}$-complete.

We note that many variations of L are also $\mathbf{EXP}^{\mathbf{NP^A}}$-complete such as running $M_n^{\mathbf{A}}$ on an empty tape.

Proof of Proposition 3.3. With the prior lemma, it suffices to show that $\mathbf{EXP}^{\mathbf{NP^A}} \subseteq \mathbf{EXP}^{\mathbf{A}}/\text{poly}$ implies $L \in \mathbf{EXP}^{\mathbf{A}}$.

Now, we know that $L \in \mathbf{EXP}^{\mathbf{A}}/\text{poly}$. Let $a(|x|)$ be the advice string, satisfying $|a(|x|)| \leq p(\ell) \in \text{poly}(\ell)$ for $|x| = \ell$ be the witness for an input x. That is, there is some Turing machine T which has

$$T(x, a(|x|)) = 1 \iff x \in L.$$

and T runs in 2^{poly} time on all inputs.

Consider the following algorithm. On input (n, x, t, i, b):

(1) Put $\ell \leftarrow |(n, x, t, i, b)|$, the length of our input.

(2) Put $\mathcal{A} \leftarrow []$, which will be a set of accepting computations.

(3) For each string y of length $\leq p(\ell)$:

  (a) Put $z \leftarrow e$, the empty string.

  (b) Repeat t times: if $T((n, x, t, z, 0), y)$ accepts, then $z \leftarrow z||0$, otherwise $z \leftarrow z||1$.

  (c) If z is the valid encoding of a computation of $M_n^{\mathbf{A}}(x)$ that accepts, push z to $\mathcal{A}$

(4) If $\mathcal{A}$ is empty, return **FALSE**

(5) Loop over $\mathcal{A}$ and, put s as the lexicographically first string in $\mathcal{A}$

(6) If the ith bit of the s is b, return **TRUE**

(7) Return **FALSE**

To see that this is a valid computation, the key point is we try every witness y, and therefore if there exists a valid computation path, we will find it. Further, for the "true" witness $y_0 = a(|x|)$, we find the lexicographically smallest path (if one exists). When y is not a valid witness, we only push z to $\mathcal{A}$ if it gives a valid computation, and this cannot be lexicographically smaller than the z generated by y_0.

Observe that step (3)(b) happens at most $t \leq 2^\ell$ times per iteration of step (3), each taking at most $O(2^\ell)$ time, and step (3) iterates at most $O(2^{p(\ell)})$ times, for a total of $O(2^{2\ell p(\ell)})$ time. Steps (1), (2), (4), (5), (6), and (7) all take at most $O(2^{p(\ell)})$ steps. Thus, $L \in \mathbf{EXP}^{\mathbf{A}}$. □

3.2 Proof of Theorem 3.1

We are now ready to prove Theorem 3.1.

Proof of Theorem 3.1. Take the oracle $\mathbf{A}$ from Proposition 3.2. Since $\oplus\mathbf{P}^{\mathbf{A}} = \mathbf{P}^{\mathbf{A}}$, we have $\mathbf{NP}^{\mathbf{A}} = \mathbf{RP}^{\mathbf{A}}$ by relativizing Valiant–Vazirani. Thus $\mathbf{NP}^{\mathbf{A}} \subseteq \mathbf{BPP}^{\mathbf{A}}$, and thus

$$\mathbf{NP}^{\mathbf{NP}^{\mathbf{A}}} \subseteq \mathbf{NP}^{\mathbf{BPP}^{\mathbf{A}}} \subseteq \mathbf{BPP}^{\mathbf{NP}^{\mathbf{A}}} \subseteq \mathbf{BPP}^{\mathbf{BPP}^{\mathbf{A}}} \subseteq \mathbf{BPP}^{\mathbf{A}}.$$

By a similar argument, we can conclude $\mathbf{PH}^{\mathbf{A}} \subseteq \mathbf{BPP}^{\mathbf{A}}$ by induction, where $\mathbf{PH}$ denotes the polynomial hierarchy.

Now, we also have

$$\mathbf{EXP}^{\mathbf{A}} \subseteq \mathbf{EXP}^{\mathbf{NP}^{\mathbf{A}}} \subseteq \mathbf{NP}^{\mathbf{A}}/\text{poly} \subseteq \mathbf{EXP}^{\mathbf{A}}/\text{poly}$$

where the middle inclusion follows from Proposition 3.2. Consequently, we have $\mathbf{EXP}^{\mathbf{A}} \subseteq \mathbf{PH}^{\mathbf{A}}$. By Proposition 3.3, we get $\mathbf{EXP}^{\mathbf{NP}^{\mathbf{A}}} \subseteq \mathbf{EXP}^{\mathbf{A}}$, and thus $\mathbf{EXP}^{\mathbf{NP}^{\mathbf{A}}} \subseteq \mathbf{PH}^{\mathbf{A}}$, and thus we are done. □

4 Proof of Proposition 3.2

We loosely follow [BT00]. In their original argument, they need to deal with double exponentially long queries, and while we must also do so, we can avoid some of the technical details by keeping track of a few other variables.

Notation 4.1. When we compute the degree of a polynomial in many variables, we take the maximum sum of the degrees of any monomial. We will work over $\mathbb{F}_2$ and therefore $-1 = 1$, which we will use throughout without note. We abuse notation and interpret oracles answers as bits and as elements of $\mathbb{F}_2$. All numbers are positive integers or elements of $\mathbb{F}_2$ unless otherwise stated.

We first prove a lemma about multilinear polynomials that compute functions.

Lemma 4.2. The function $\mathrm{OR} : \mathbb{F}_2^m \to \mathbb{F}_2$ and $\mathrm{AND} : \mathbb{F}_2^m \to \mathbb{F}_2$ can be written as (multi-)degree m multilinear polynomials but cannot be written as smaller (multi-)degree multilinear polynomials.

Proof. Observe that multilinear polynomials must be unique as otherwise their difference is the 0 function, which cannot be multilinear unless it is the zero polynomial.

Thus, observe

$$\mathrm{AND}(x_1, \ldots, x_m) = \prod x_i \quad \text{and} \quad \mathrm{OR}(x_1, \ldots, x_m) = 1 - \prod (1 + x_i). \qquad \square$$

Remark 4.3. This lemma is useful because if we have some function that is a) multilinear, and b) is minimal in the sense that any proper subset cannot satisfy some property, then we can, in some sense, show that this function is AND or OR of these properties and consequently bound the number of such variables.

We are now ready to prove Proposition 3.2.

Our approach is to construct a set A while continually ensuring the following conditions

(1) for various string $s \in \{0,1\}^*$, we assign some polynomial p_s over $\mathbb{F}_2$ with many variables y_*, b_*;

(2) at various points, we will also assign values to the above variables, and once any p_z's variables have been fully defined, we require $p_z(y_*, b_*) = 1 \iff z \in A$.

(3) $\deg p_z \le |z|^2$.

Note that if p_z is not fully specified, we do not force either $z \in A$ or $z \notin A$. Note that (3) will allow us to find a (quadratic length) witness, ensuring that $\mathbf{EXP}^{\mathbf{NP^A}} \subseteq \mathbf{NP^A}/\mathrm{poly}$.

We will put elements in $\mathbf{A}$ or the complement $\mathbf{A}^c$ exactly when (2) requires us to. First, by padding our witnesses and queries, we know there exists a constant $C \gg 1$ and oracle Turing machines

- M such that for every oracle $\mathbf{A}$, $M^{\mathbf{A}}(x) \equiv 1 \pmod 2$ if and only if $x \in L^{\mathbf{A}}$ makes $L^{\mathbf{A}}$ a $\oplus\mathbf{P}^{\mathbf{A}}$-complete family. In addition, M runs in $C|x|$ time.
- K such that $K^{\mathbf{A}}$ is a $\mathbf{NP}^{\mathbf{A}}$ verifier. In addition, K runs in $C|x|$ time.
- $\mathcal{E}$ such that $\mathcal{E}^{\mathbf{NP^A}}(x) = 1 \iff x \in H^{\mathbf{A}}$ defines a $\mathbf{EXP}^{\mathbf{NP^A}}$-complete family. In addition, $\mathcal{E}$ runs in $C2^{|x|}$ time.

We now enumerate our desired conditions for $\mathbf{A}$. Therein, strings in $\mathbf{A}$ will be of the form

1. (Type 0; ensuring $\mathbf{P}^{\mathbf{A}} = \oplus\mathbf{P}^{\mathbf{A}}$) When $w \in L^{\mathbf{A}}$, we will include $(0, w, 1^{C^2|w|^2})$. This will ensure that we can easily query an $\oplus\mathbf{P}^{\mathbf{A}}$ complete problem in deterministic polynomial time.
2. (Type 1; ensuring $\mathbf{EXP}^{\mathbf{NP}^{\mathbf{A}}} \subseteq \mathbf{NP}^{\mathbf{A}}/\text{poly}$) When $w \in H^{\mathbf{A}}$, we will include $(1, w, f(|w|), v)$ for at least one v of length $C^2|w|^2$. Here, f will be our quadratically bounded advice function.

Observe that if we can ensure these two conditions, the propositions follows. The advantage of this approach is that we can use M and $\mathcal{E}$ instead of L and H, which is beneficial because their time-bounds are well-behaved. We now define p_s for each s.

- For strings s that are neither type 0 or type 1, define $p_s = 0$. These satisfy (3) (we need not worry about defining the degree of the 0 polynomial).
- For each type 0 string $s = (0, w, 1^{C|w|^2})$ let $\mathcal{C}$ be the set of accepting computations of $M^{\mathbf{A}}(x)$. Then put

$$p_s = \sum_{\rho \in \mathcal{C}} \prod (p_{\rho,i} + b_{\rho,i} + 1), \tag{1}$$

  where $p_{\rho,i}$ is p_t where t is the ith oracle query along the computation path, and $b_{\rho,i}$ is the output. By Claim 4.4, these will satisfy (3).
- For each type 1 string of the form $s = (1, w, x, z)$, we will define some $y_s = y_{(1,w,x,z)}$, and put p_s as the polynomial y_s. These also satisfy (3).

Claim 4.4. For each type 0 string of the form $(0, x, 1^{|x|^2})$ we have $p_x \equiv \#M^{\mathbf{A}}(x) \pmod 2$ and p_x has degree at most $|x|^2$.

Proof. Proceed by induction on $|x|$. Assume the statement holds for strings of length $< n$. On an input of length n we can only query at most Cn strings. If we query some w, then we have $|w|^2 \leq Cn$ and thus $|w| \leq \sqrt{n/C}$. By induction, these p_w have degrees bounded by $(\sqrt{n/C})^2 = \frac{n}{C}$, and thus each summand in the right side of (1) has degree bounded by $\frac{n}{C} \cdot Cn = n^2$. □

We now proceed with step $n = 0, n = 1, n = 2, \ldots$, for each non-negative integer n. At stage n, we will ensure that all inputs of length n behave as we desire. Individually, this is not hard to achieve; the difficulty lies in making sure fixing a given input does not change the computation path of any other previously fixed input. To do so, we do the following.

For each string x of length n, running $\mathcal{E}(x)$ will query some strings. There are two possibilities. There either is some $\mathbf{A}' \supseteq \mathbf{A}$ such that $q \in K^{\mathbf{A}'}$ for all prior queries q, or there is no such $\mathbf{A}'$. In the latter case, we will forbade $q \in \mathbf{A}$, and in the former case we will attempt to add as few strings to $\mathbf{A}$ as possible. This will cause $q \in K^{\mathbf{A}}$ to never be able to change (i.e. since if adding some strings into A can allow $q \in K^{\mathbf{A}}$, then we will do so, and otherwise we will never be able to do so).

Given that we only need to accept along one path, fix some computation path of $K^{\mathbf{A}}$. Let q_i be a set of queries along this path and b_i their responses. Put

$$P_q = \prod_i (p_{q_i} + b_i + 1).$$

Now, by Claim 4.4, $\deg p_{q_i} \leq (2^n)^2$, and thus the degree of P_q is bounded by $O(2^{3n})$, as we can only make $O(2^n)$ queries. Now, if we pick the set of strings to add to $\mathbf{A}$ to be locally minimal, we know that removing any one of the strings, must cause our computation to change its answer. Thus, we are computing the AND of all the strings we wish to add (or the negation), and thus by Lemma 4.2, we conclude that we need to add at most $\deg P_q \leq O(2^{3n})$ many strings.

Now, since we have added at most $O(2^{3n}) \cdot 2^n$ strings at stage n, we know that $\#\mathbf{A}$ at the end of (this part of) stage n is bounded by $O(2^{4n})$, and thus there is some string $w = w_n$ of length n^2 such that for all v, v' we have $(1, w, v, v') \notin \mathbf{A}$. We will take w as our witness for inputs of length n.

We claim this can be done consistently for every x with $\mathcal{E}^{K^{\mathbf{A}}}(x) = 1$. Suppose not, consider

$$Q_x \;=\; 1 - \prod_{\mathbf{A}' \supseteq \mathbf{A}} P_q,$$

which takes on value 0 and has degree bounded by 2^{9n}. Pick $\mathbf{A}'$ such that not subextension of $\mathbf{A}$ suffices. Thus, removing any single string from $\mathbf{A}'$ causes Q_x to become 0. Thus, we must be computing the OR of the variables that make up Q_x, which are some y_*. But there are at least 2^{n^2} such y_* so Lemma 4.2 says Q_x is degree at least $2^{n^2} > 2^{9n}$, contradiction.

Thus, at the end of stage n, we successfully compute the desired answer for all input strings of length $\leq n$ and no query made on a string of length $\leq n$ can ever its answer changed on a later step. Continuing this process infinitely, and taking the union of the $\mathbf{A}$ we get at each stage, we get some oracle with our desired properties. This completes the proof of Proposition 3.2. □

Acknowledgements

This paper was written for MIT's Advanced Complexity Theory (18.405). The author thanks Ryan Williams for helpful comments and feedback as well as various pointers during the project's early stages.

References

[AKS04] Manindra Agrawal, Neeraj Kayal, and Nitin Saxena. "PRIMES is in P". In: *Annals of Mathematics* 160.2 (2004), pp. 781–793. DOI: 10.4007/annals.2004.160.781.

[Bac90] Eric Bach. "Explicit Bounds for Primality Testing and Related Problems". In: *Mathematics of Computation* 55.191 (1990), pp. 355–380. DOI: 10.1090/S0025-5718-1990-1023756-8.

[BG81] Charles H. Bennett and John Gill. "Relative to a Random Oracle A, $P^A \neq NP^A \neq$ co-NP^A with Probability 1". In: *SIAM Journal on Computing* 10.1 (1981), pp. 96–113.

[BGS75] Theodore Baker, John Gill, and Robert Solovay. "Relativizations of the $P =?\ NP$ Question". In: *SIAM Journal on Computing* 4.4 (1975), pp. 431–442. DOI: 10.1137/0204037.

[BH92] Harry Buhrman and Steven Homer. "Superpolynomial Circuits, Almost Sparse Oracles and the Exponential Hierarchy". In: *Foundations of Software Technology and Theoretical Computer Science*. Ed. by R. Shyamasundar. Vol. 652. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer, 1992. ISBN: 978-3-540-56287-0. DOI: 10.1007/3-540-56287-7_99.

[BT00] Harry Buhrman and Leen Torenvliet. "Randomness is Hard". In: *SIAM Journal on Computing* 30.5 (2000), pp. 1485–1501. DOI: 10.1137/S0097539799360148. eprint: https://doi.org/10.1137/S0097539799360148. URL: https://doi.org/10.1137/S0097539799360148.

[LP19] Hendrik W. Lenstra and Carl Pomerance. "Primality Testing with Gaussian Periods". In: *Journal of the European Mathematical Society* 21.4 (2019), pp. 1229–1269. DOI: 10.4171/JEMS/861.

[Rab80] Michael O. Rabin. "Probabilistic Algorithm for Testing Primality". In: *Journal of Number Theory* 12.1 (1980), pp. 128–138. DOI: 10.1016/0022-314X(80)90084-0.

[Sch80] Jacob T. Schwartz. "Fast Probabilistic Algorithms for Verification of Polynomial Identities". In: *Journal of the ACM* 27.4 (1980), pp. 701–717. DOI: 10.1145/322217.322225.